

Partitioned Hash Join with Duplicates

A Distributed MPI Study

Gabriele Marsili
SPM a.y. 2025/26

1 Introduction

This report describes the distributed implementation of the partitioned hash join developed throughout the previous modules. The algorithm is unchanged: R and S are partitioned through a common key-mapping function and a local join is performed independently on each partition. Viewed as a structured pattern, the distributed pipeline is a Map with a partitioned-state local reduction followed by a global All-Reduce: each rank applies the same build/probe kernel to its share of the data, computes a local triple ($join_count, ck_1, ck_2$), and the three values are then combined across ranks with a single collective. The aim of this module is to study how communication costs interact with local computation when memory bandwidth and memory capacity are spread across multiple cluster nodes, and to compare the resulting performance with the shared-memory variants of Modules 2 and 3.

Two implementations are presented. They share the same algorithmic skeleton and the same generator scheme as the previous modules: a pure MPI version that exchanges partitions via `MPI_Alltoallv`, and a hybrid MPI+OpenMP version in which *every* per-rank phase is parallelised with OpenMP, so that a single rank can drive all the cores of a node. All experiments were carried out on the SPM cluster (8 compute nodes, dual-socket Intel Xeon E5-2640 v2, 32 logical CPUs and 128 GB of RAM per node, Open MPI 5.0.3).

2 Distributed Algorithm

2.1 Partitioning and Rank Assignment

The mapping function from Modules 1–3 is unchanged. A 64-bit key k is folded to 32 bits and multiplied by the Fibonacci constant, $p = ((k_{lo} \oplus k_{hi}) \cdot \varphi_{32}) \gg (32 - \log_2 P)$. Each partition is assigned to one of the R MPI ranks by $dest(p) = p \bmod R$ with local index $\ell = p \div R$ in $[0, P/R)$. The constraint $P \bmod R = 0$ keeps the number of partitions per rank exactly balanced.

2.2 Pipeline

Each rank generates only its slice of R and S using `splitmix64` skip-ahead so the concatenation of the local slices equals the sequential generator output byte-for-byte. The distributed pipeline has eight measured phases: *histogram_local*, *scatter_local* (in dest-major order so each rank’s send buffer is naturally contiguous per destination), *comm_sizes* (`MPI_Alltoall` of the per-rank

counts), *comm_payload* (`MPI_Alltoallv` of the records), *histogram_post* and *scatter_post* (local re-layout of the received buffer so the M3 build/probe kernel can be reused unchanged), *join_local*, and the final `MPI_Allreduce` of the three aggregates ($join_count, ck_1, ck_2$). Each interval is preceded by an `MPI_Barrier` so the rank-local time reflects the actual critical path; the reduction takes the maximum across ranks so the breakdown shows the slowest path per phase.

2.3 Choice of the Redistribution Primitive

`MPI_Alltoallv` was chosen as the canonical primitive for total-exchange of partitioned data (lez. 25). A non-blocking alternative based on `MPI_Isend/Irecv` was considered: it gives more control over communication-computation overlap, but the post-exchange phases (*histogram_post*, *scatter_post*, build and probe) all depend on the received buffer, leaving no independent local work to hide the collective behind. The breakdown measurements of §4.4 also confirm that the collective implementation already saturates the network bandwidth on the cluster. A small `MPI_Alltoall` of the per-rank counts precedes each payload exchange so the receive buffers can be sized exactly. The final aggregate uses `MPI_Allreduce` (a tree-based reduction with $O(\log R)$ rounds, lez. 25) on three 64-bit values, whose cost is below a millisecond at every rank count tested.

2.4 Hybrid MPI+OpenMP

The hybrid implementation uses one MPI rank per node and `OMP_NUM_THREADS` equal to the number of logical CPUs. Every local phase is OpenMP-parallel: *histogram_local* and *histogram_post* use per-thread private histograms merged at the end, *scatter_local* and *scatter_post* reuse those per-thread histograms to compute per-thread offsets and then write into the output buffer with static scheduling, and *join_local* runs a `parallel for` with `schedule(dynamic)` and a sum reduction. The dynamic schedule on the join mirrors M3’s loop variant under skewed workloads (lez. 14): partition cardinalities vary and a static split would stall the last threads on the largest partitions. MPI is initialised with `MPI_THREAD_FUNNELED`: only the main thread enters MPI, which is the standard choice for funneled hybrid codes because `MPI_THREAD_MULTIPLE` requires the implementation to serialise internal critical sections and pays a measurable overhead even when the application never issues concurrent MPI calls.

3 Validation

Three levels of verification were applied. For tiny inputs ($N_R, N_S \leq 500$) rank 0 regenerates the full relations and compares the global ($join_count, ck_1, ck_2$) to an $O(N^2)$ naive join. For larger uniform inputs the same triple is compared to the sequential baseline of Module 3 across rank counts $R \in \{1, 2, 4, 8\}$ on the laptop and $\{32, 64, 128\}$ ranks on the cluster. The sequential binary now accepts `-skew` and `-hot`, so the same triple comparison covers the skewed case as well. All 36 local cases and all 18 cluster cases passed; the validation logs are stored in `results/validation_local.log` and `results/cluster/validation_cluster.log`.

4 Experimental Evaluation

4.1 Setup

Strong-scaling, weak-scaling and breakdown runs use $P = 256$ partitions, seed 42 and $max_key = N_R/2$, with five repetitions per cell and the median time reported. Strong scaling fixes the global input at $N_R = 50$ M and $N_S = 100$ M records; weak scaling holds the per-rank workload constant at $N_R/R = 2$ M records (with $N_S/R = 4$ M). Pure MPI uses one rank per logical CPU (32 ranks per node); the hybrid uses one rank per node with `OMP_NUM_THREADS=32`, `OMP_PROC_BIND=close` and `OMP_PLACES=cores`. The sequential baseline used for speedup is the Module 3 sequential binary, recompiled with `-march=ivybridge` (the compute-node ISA, which does not support the AVX2 produced by `-march=native` on the Haswell login node) and timed on a dedicated compute node. Two reference times are used, one per workload: $t_{seq}^{unif} = 4.52$ s on uniform input and $t_{seq}^{skew} = 2.46$ s on the skewed input ($\rho = 0.9$, four hot keys), both for $N_R = 50$ M, $N_S = 100$ M. The skewed baseline is substantially lower because four hot keys collapse the join output to about 36 M matches versus 200 M for uniform, shrinking the join phase accordingly.

4.2 Strong Scaling

Figure 1 reports the speedup as a function of node count. The hybrid variant is the clear winner from two nodes upwards on both workloads. On the uniform dataset the hybrid speedup grows monotonically from $2.8\times$ at one node to $12.2\times$ at eight, crossing the ideal N -node line because the per-node cost is dominated by collective communication that does not grow as fast as the rank count. Pure MPI reaches $7.7\times$ at two nodes (64 ranks) and $7.1\times$ at eight (256 ranks), but at four nodes (128 ranks) it collapses to $3.1\times$. The collapse is not measurement noise in the strict sense, but it is bimodal across the five repetitions: the minimum is 0.68 s (in line with the two-node extrapolation), the median is 1.48 s and four of five runs sit in the slow mode. We interpret this as Open MPI’s `Alltoallv` algorithm selection at 128 ranks: in the slow mode the collective hits a routing pattern that the cluster fabric handles poorly, and the choice is not under our control without injecting MCA tuning parameters. The fact that both the 0.68 s fast mode

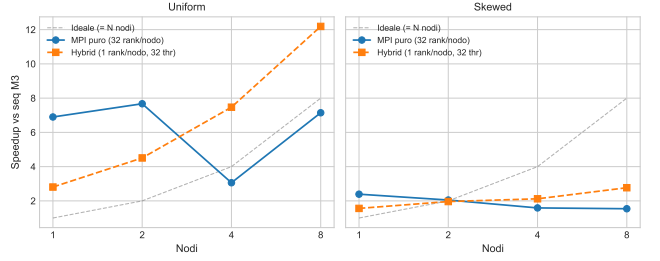


Figure 1: Strong scaling on 1, 2, 4, 8 nodes. The dashed grey line marks the ideal N -node speedup. $N_R = 50$ M, $N_S = 100$ M, $P = 256$, five repetitions, median reported.

and the eight-node 0.63 s runs are stable shows that the bandwidth is there; the median reflects how often the fast mode is actually picked.

Under the skewed dataset, measured against the dedicated skewed baseline of 2.46 s, the curves separate further. Pure MPI degrades monotonically from $2.4\times$ at one node to $1.5\times$ at eight: every additional rank adds entries to the `Alltoallv` fan-out, but the hot partitions concentrate on a small number of receivers, so the collective waits on the slowest ranks. The hybrid version still grows, from $1.6\times$ at one node to $2.8\times$ at eight; with at most eight participants the fan-out stays small and the work per rank is large enough that load imbalance dominates rather than communication latency. The smaller absolute speedups compared to the uniform case are partly arithmetic: the skewed baseline is $1.8\times$ faster than the uniform one to begin with, so the distributed pipeline has less serial work to amortise.

4.3 Weak Scaling

Figure 2 shows the weak-scaling efficiency t_1/t_N for the uniform input with a per-rank workload of 2 M records of R against 4 M of S . The two implementations behave very differently: the hybrid variant retains 0.57 efficiency at eight nodes, while pure MPI drops to 0.20. The reason lies in the rank-count growth of the collective. With one rank per node the `Alltoallv` fan-out goes from 8 pairs on a single node to 64 on eight, and the median payload time grows mildly from 15 ms to 56 ms, a $3.7\times$ increase against an $8\times$ growth of the global volume. With 32 ranks per node the global rank count grows from 32 to 256 and the same collective becomes the dominant cost: the median payload time goes from 0.12 s to 2.99 s, a $25\times$ increase. The trend follows the textbook $\alpha + \beta N/R$ model for collectives (lez. 23): once βN saturates the bisection bandwidth, the remaining variable is the latency α , which scales with the total number of participants.

4.4 Phase Breakdown

Figure 3 stacks the eight measured intervals for the four configurations on four nodes. The bars also serve as a direct cost comparison between the two implementations on the same workload: hybrid uniform totals about 0.52 s, pure MPI uniform about 1.48 s, with the gap accounted almost entirely by `comm.payload`. Pure MPI on uniform spends 92% of the wall time in the `Alltoallv` of 128

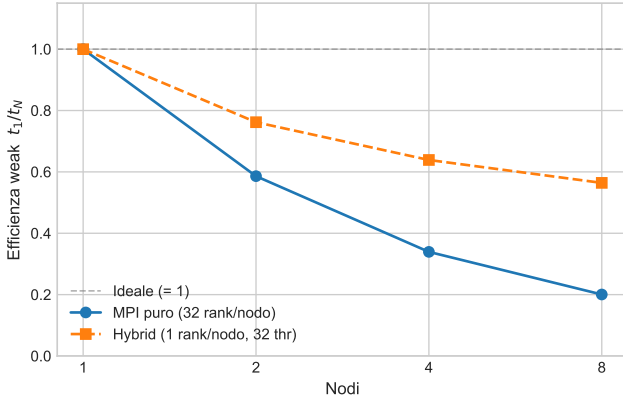


Figure 2: Weak-scaling efficiency t_1/t_N . Per-rank workload constant at $2\text{M} \times 4\text{M}$ records; $P = 256$.

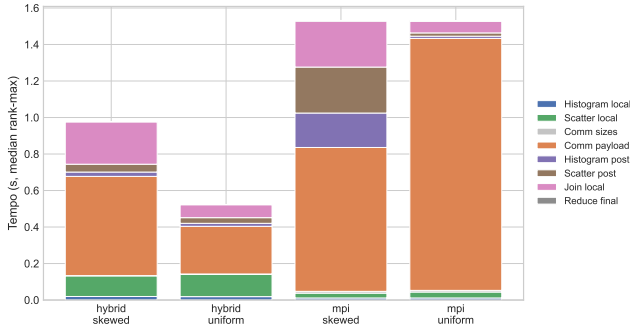


Figure 3: Median rank-max breakdown for the four configurations on four nodes (five repetitions, $N_R = 50\text{M}$, $N_S = 100\text{M}$, $P = 256$).

ranks, which is bandwidth-bound on the cluster network; the four local phases together account for around 0.07s because each rank only handles a $1/128$ slice of the input. The hybrid variant inverts the per-phase distribution: the local phases (now OpenMP-parallel) are a few tens of milliseconds each, while *comm_payload* becomes the largest contribution at 44% of the wall time, even though the eight-rank collective is much cheaper in absolute terms than the 128-rank one. The skewed pure-MPI run spreads the cost roughly evenly between communication (50%) and the post-*Alltoallv* phases, reflecting load imbalance on a few hot ranks. The skewed hybrid is the worst case for the local join, since the hot partitions cluster on three or four ranks out of eight.

4.5 Skewed Workload

Figure 4 compares the total time under uniform and skewed inputs across node counts. Hybrid times rise modestly under skew (a roughly constant offset of $0.2\text{--}0.5\text{s}$) while pure-MPI skewed times grow monotonically with N instead of shrinking. The distributed setting amplifies the load imbalance that the M3 task variant could absorb on a single shared-memory node. The underlying issue is the one a partitioned-state farm always faces when the input distribution is skewed: hashing balances the *number* of partitions perfectly across ranks, but it cannot balance the *volume* of records per partition, so a few heavy partitions concentrate on a handful of re-

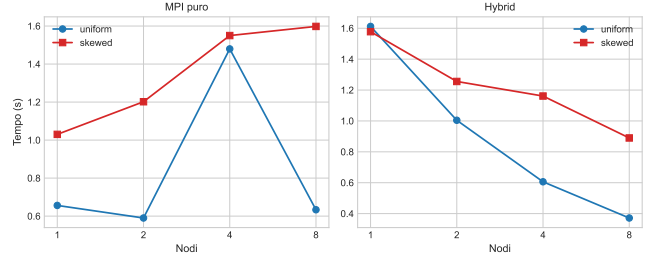


Figure 4: Total time under uniform vs. skewed input ($\rho = 0.9$, 4 hot keys).

ceivers and the global step waits on them. When the hot partitions live on three or four ranks out of 256, the final *Allreduce* pays the cost of the most-loaded rank, and that rank's local join takes the same time it would on a single-node OpenMP run. A partition-aware mapping that splits the heaviest partitions across multiple ranks (so the per-rank work is bounded by the average rather than the maximum) would mitigate this and is left as future work.

5 Discussion

On a single node the pure-MPI run with 32 ranks completes in 0.66s (speedup $6.9\times$ over the sequential M3 baseline), while the hybrid with one rank and 32 threads takes 1.61s ($2.8\times$). The gap closes from two nodes onwards: hybrid takes 1.00s at two nodes vs. 0.59s for MPI, and from four nodes onwards hybrid is faster than pure MPI both in absolute terms (0.61s vs. 1.48s) and in worst-case predictability. The cross-over is driven by the *Alltoallv* cost: pure MPI keeps the local phases cheap by splitting them across 32 ranks per node, but pays an ever-increasing communication tax as the global rank count grows; the hybrid amortises a small parallelisation overhead in the local phases against a much cheaper collective.

Relative to Module 3's OpenMP loop variant the hybrid M4 binary matches the multi-thread shared-memory result at one node (the local pipeline is the same as in M3, plus an MPI no-op), and from two nodes onwards extends the curve into the distributed-memory regime, where M2 and M3 cannot follow at all.

Conclusions

The distributed implementation preserves the algorithmic structure of the previous modules and re-uses the M3 build/probe kernel unchanged. Within the eight-node budget of the cluster the hybrid pipeline reaches a $12.2\times$ speedup over the sequential M3 baseline on uniform input with 0.57 weak-scaling efficiency, beating the $7.7\times/0.20$ obtained by pure MPI. The breakdown confirms that further speedup is communication-bound: *Alltoallv* is the largest contribution to wall time in every multi-node configuration. The hybrid variant pays a non-trivial single-node cost for parallelising the local phases with OpenMP, but it amortises that cost across nodes and is preferable from two nodes onwards.

Addendum: Hybrid MPI+OpenMP

The hybrid configuration uses one MPI rank per node with `OMP_NUM_THREADS=32`, `OMP_PROC_BIND=close` and `OMP_PLACES=cores`. Every per-rank phase is OpenMP-parallel: *histogram_local* and *histogram_post* build per-thread private histograms and merge them at the end, *scatter_local* and *scatter_post* reuse those per-thread tables to derive per-thread offsets so writes never collide on the same slot of the output buffer, and *join_local* runs a `parallel for` with `schedule(dynamic)` and a sum reduction. The static scheduling on the scatter passes mirrors the histogram pass that fed it, so the same input range is visited by the same thread in both phases.

Effect on the single-node case. A first version of the hybrid parallelised only *join_local* and left the other four local phases single-threaded. That implementation took 6.25 s on one node, slower than the sequential M3 baseline, because the only rank had to scan all 150 M records through one CPU pipeline before reaching the parallel join. Parallelising all local phases brings the one-node hybrid down to 1.61 s, a $3.9\times$ improvement, and is what makes the multi-node curve of Figure 1 monotonic. The lesson here is Amdahl (lez. 10): in a hybrid pipeline the sequential phase becomes the bottleneck, and parallelising the obvious hot loop is not enough on its own.

Collective cost. Going from 256 ranks (pure MPI on 8 nodes) to 8 ranks (hybrid on 8 nodes) reduces the median `Alltoallv` time on uniform input from 0.53 s to 0.18 s, a $3\times$ speedup with the same total payload moved. `Allreduce` of the three aggregates is consistently below a millisecond regardless of node count because it sees only 1 to 8 participants. The collective cost still dominates the wall time of the hybrid (around 44% at four nodes and growing with N), which suggests that the next optimisation to consider is communication-computation overlap: posting the `Alltoallv` as a non-blocking `MPI_Ialltoallv` while the local build step starts on the partitions that are already known to be local.

Rank-per-socket layout. The compute nodes have two NUMA domains; binding one rank per socket (two ranks per node, 16 threads each) would let the local join run entirely within one NUMA domain at the cost of doubling the `Alltoallv` fan-out. Given that the local phases are now parallel and the wall time is communication-bound, doubling the rank count is unlikely to help; the experiment was not run within the time budget.